

PART-B

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

Selenium is an open-source tool used for automating **web browsers**. It is mainly used to test web applications by performing actions like opening a website, clicking buttons, entering data, and checking results.

Suitable use cases:

1. Testing an e-commerce website, such as login, search, and checkout.
2. Testing a web-based banking application.

Unsuitable use cases:

1. Testing desktop applications such as MS Word or Calculator.
2. Automating mobile apps directly. For mobile app automation, tools like Appium are normally used.

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

Tool	Main purpose
Selenium WebDriver	Used to write and run browser automation scripts using languages such as Python, Java, C#, etc.
Selenium IDE	A browser extension used mainly for recording and playing back browser actions. It is easier for beginners but less flexible than WebDriver.
Selenium Grid	Used to run tests on multiple browsers, machines, or operating systems, often in parallel.

Q3. Explain the roles of the test script, Selenium client library, browser driver, browser, and WebDriver protocol during one command such as driver.get().

The **Selenium client library** provides the Python methods such as driver.get() and converts the Python instruction into a WebDriver request.

The request is sent through the **WebDriver protocol** to the appropriate **browser driver**.

The **browser driver** communicates with the actual **browser** and tells it to navigate to the requested URL.

The browser then opens the page and the result is returned through the driver back to the Selenium client.

So the basic flow is:

Python test script → Selenium client library → WebDriver communication → Browser Driver → Browser

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

A virtual environment creates a separate Python environment for a particular project.

It is recommended because different projects may require different versions of the same package. For example, one project might need an older Selenium version while another needs a newer one.

If packages are installed globally, upgrading or changing a package for one project can affect another project and may cause **dependency conflicts**.

For example, one project could stop working because a globally installed package was upgraded.

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

Selenium Manager is a tool that comes with modern Selenium releases and helps manage the required browser drivers automatically.

Earlier, we commonly had to download the correct ChromeDriver or another browser driver manually and specify its location in the code.

With Selenium Manager, in many normal cases we can simply write:

```
from selenium import webdriver
```

```
driver = webdriver.Chrome()
```

Selenium can detect the browser and manage the appropriate driver automatically.

This makes setup much easier and reduces problems related to manually downloading and maintaining drivers.

Q6. Differentiate `driver.find_element()` and `driver.find_elements()`. What is returned when no element matches?

`find_element()` is used when we want to find **one element**.

```
element = driver.find_element(By.ID, "username")
```

If no matching element is found, it raises a **NoSuchElementException**.

`find_elements()` is used when we want to find **multiple elements**.

```
elements = driver.find_elements(By.TAG_NAME, "input")
```

If there are no matching elements, it returns an **empty list []** instead of throwing `NoSuchElementException`.

Q7. Compare ID, name, class name, CSS selector, and XPath locators. Which locator strategy would you prefer for stable tests, and why?

Different locators identify elements in different ways:

- **ID:** Finds an element using its id. Usually a very good choice if the ID is unique and stable.

- **Name:** Uses the element's name attribute.
- **Class name:** Uses the CSS class of the element, but can be less reliable if classes are dynamically generated or frequently changed.
- **CSS selector:** Provides a flexible way to locate elements using CSS syntax.
- **XPath:** Can locate elements using their structure, attributes, text, and relationships with other elements.

For stable tests, I would generally prefer a **unique and stable ID first**. If a stable ID isn't available, I would usually use a **stable CSS selector**, and XPath when the element requires more complex relationships or conditions.

The important thing is not simply choosing one locator everywhere. The locator should depend on how stable the application's HTML is.

Q8. Differentiate implicit wait, explicit wait, and time.sleep(). Give one appropriate use case for each.

Implicit wait tells Selenium to wait for a certain amount of time when trying to find elements.

```
driver.implicitly_wait(10)
```

It can be useful when elements generally take a short amount of time to appear throughout the application.

Explicit wait waits for a specific condition.

```
WebDriverWait(driver, 10).until(
    EC.visibility_of_element_located((By.ID, "username"))
)
```

This is usually preferred when we know what condition we are waiting for, such as an element becoming visible or clickable.

time.sleep() simply pauses the Python program for a fixed amount of time.

```
time.sleep(3)
```

It can be useful for a quick debugging experiment, but it is generally not preferred in real automation because the application may become ready earlier or later than the fixed time.

Q9. Why should `driver.quit()` normally be placed in a finally block or teardown method? How is it different from `driver.close()`?

`driver.quit()` should be used to properly end the Selenium session and close **all browser windows** opened by that WebDriver session.

Putting it in a finally block or test teardown makes sure it runs even if the test fails due to an exception.

For example:

try:

```
driver.get("https://example.com")
```

finally:

```
driver.quit()
```

`driver.close()` only closes the **currently active browser window/tab**.

So:

- `close()` → closes the current window/tab.
- `quit()` → ends the WebDriver session and closes all associated windows.

Q10. What causes `StaleElementReferenceException`? Describe two robust ways to handle it without hiding a genuine application defect.

This exception usually happens when we found an element earlier, but the page changed and the old element reference is no longer attached to the current DOM.

For example, a page may refresh or JavaScript may replace an element after we initially located it.

Two good ways to handle it are:

1. Locate the element again before using it.

Instead of keeping an old WebElement reference for too long, find the element again after the page changes.

2. Use an explicit wait and locate the element after the expected DOM update.

For example, wait for the new element to become available and then interact with it.

We should not simply catch the exception and ignore it because that could hide a real application problem.

Q11. What is headless browser execution? Identify two benefits and two limitations.

Headless execution means running a browser **without displaying its graphical user interface**.

For example, Chrome can run in headless mode while Selenium still interacts with the website.

Benefits:

1. It can use fewer system resources.
2. It is useful for running automated tests on CI/CD servers where there may be no graphical desktop.

Limitations:

1. Some browser behavior can differ from normal visible execution.
 2. Debugging can be harder because we cannot directly see what the browser is doing.
-

Q12. Explain how browser and driver version mismatch can affect a Selenium session. Is this still relevant when Selenium Manager is used? Justify.

The browser driver needs to be compatible with the browser it controls. If there is an incompatible version, Selenium may fail to create a browser session or may show a **session creation/version compatibility error**.

Selenium Manager has reduced this problem significantly because it can automatically discover the browser and manage the appropriate driver in normal supported setups.

However, version compatibility is **still relevant**. Selenium Manager doesn't mean that version problems can never occur. Issues can still happen because of unusual environments, manually managed drivers, unsupported browser setups, network restrictions, or other configuration problems.

So Selenium Manager makes driver management easier, but it doesn't completely eliminate the concept of browser-driver compatibility.

Q13. Why can a script work on one computer but fail on another? Discuss at least four environment-related reasons.

There can be several reasons:

1. **Different browser versions** — Chrome, Firefox, or Edge may be different versions.
2. **Different Selenium/Python versions** — the script may depend on a particular package version.
3. **Different operating systems** — Windows and Linux can behave differently in some cases.
4. **Different browser drivers or driver configuration** — especially when drivers are manually managed.
5. **Different screen resolution or display settings** — this can affect some UI interactions.
6. **Network or proxy differences** — one computer may have access to a website while another doesn't.

7. **Missing dependencies or environment variables** — required packages or configurations may not be present.

So automation tests should ideally run in a controlled and reproducible environment.

Q14. What information should be collected before reporting a Selenium defect so another person can reproduce it?

I would collect enough information for another tester or developer to reproduce the problem.

Important information includes:

- Test case name and purpose
- Exact steps to reproduce the issue
- Expected result
- Actual result
- URL or application environment
- Browser and browser version
- Operating system
- Python and Selenium versions
- Error message and complete stack trace
- Screenshots or video if useful
- Test data used
- Relevant test code
- Timestamp and test execution details

If possible, I would also mention whether the problem happens **every time or only occasionally**.

Q15. Identify three ethical or security precautions required when automating a website.

Three important precautions are:

1. **Get proper permission before automating a website**, especially if it is not our own application or if the automation could create significant traffic.
2. **Protect sensitive information**, such as usernames, passwords, API keys, and personal data. These should not be hard-coded or exposed in logs.
3. **Avoid excessive requests or aggressive automation**, because it can put unnecessary load on the website or affect other users.

Part C — Installation Challenges to Solve

Complete Technical Diagnostics, Root Cause Analysis & Justified Solutions (30 Marks)

Task Domain: Selenium Test Automation Total Challenges: 10 (C1 to C10)

Evaluation: 30 Marks (3 Marks / Challenge)

C1 pip is not recognized

OBSERVED ERROR MESSAGE:

```
'pip' is not recognized as an internal or external command, operable program or batch file. (Windows)
zsh: command not found: pip (macOS / Linux)
```

LIKELY ROOT CAUSE:

The directory housing the `pip` executable (e.g., `PythonXX\Scripts\` on Windows or `/usr/local/bin` on POSIX) is missing from the system's `PATH` environment variable, or Python was installed without enabling "Add Python to PATH".

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
python --version
python -m pip --version
echo %PATH%          # Windows Command Prompt
echo $PATH           # macOS / Linux
```

SOLUTION ATTEMPTED:

Appended Python's installation directory and `Scripts` path (e.g., `C:\Users\<User>\AppData\Local\Programs\Python\Python311\Scripts`) to the User `PATH` environment variable. As an immediate workaround, executed pip directly via the interpreter module flag: `python -m pip install <package>`.

RESULT AFTER THE FIX:

`pip --version` returns the active pip package version and linked Python environment path without shell errors.

SCREENSHOT OR LOG EVIDENCE:

```
C:\Users\Admin> pip --version
pip 24.0 from C:\Users\Admin\AppData\Local\Programs\Python\Python311\Lib\site-packages\pip (python
3.11)
```

One Preventive Recommendation: Always check the "Add python.exe to PATH" checkbox during Windows Python installer setups, and standardize build scripts to invoke package operations via `python -m pip`.

C2 Package installed, import fails

OBSERVED ERROR MESSAGE:

```
ModuleNotFoundError: No module named 'selenium'
```

LIKELY ROOT CAUSE:

Python interpreter mismatch. The package was installed into the global base Python environment via CLI, but the IDE (VS Code / PyCharm) is running tests against an isolated virtual environment (`.venv`) where Selenium is not installed.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# In IDE terminal vs system terminal:
which python    # or (Get-Command python).Path on Windows
python -c "import sys; print(sys.executable); print(sys.path)"
pip list
```

SOLUTION ATTEMPTED:

Set the IDE interpreter path specifically to `./.venv/bin/python` (or `./.venv/Scripts/python.exe`). Alternatively, activated the project virtual environment before installing:

```
source .venv/bin/activate # Windows: .venv\Scripts\activate
python -m pip install selenium
```

RESULT AFTER THE FIX:

```
python -c "import selenium; print(selenium.__version__)" resolves cleanly without throwing
ModuleNotFoundError.
```

SCREENSHOT OR LOG EVIDENCE:

```
(.venv) project$ python -c "import selenium; print(selenium.__file__)"
/Users/admin/project/.venv/lib/python3.11/site-packages/selenium/__init__.py
```

One Preventive Recommendation: Enforce dedicated virtual environments (`python -m venv .venv`) and bind workspace configuration files (`.vscode/settings.json`) to use the repository's local environment interpreter.

C3 Driver/browser mismatch

OBSERVED ERROR MESSAGE:

```
SessionNotCreatedException: Message: session not created: This version of ChromeDriver only supports
Chrome version 114; Current browser version is 126.0.6478.127
```

LIKELY ROOT CAUSE:

A legacy, hardcoded `chromedriver` binary located in the system `PATH` or project folder overrides Selenium 4's native Selenium Manager, causing a major version mismatch when the browser auto-updates.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
where chromedriver # Windows
which -a chromedriver # macOS / Linux
google-chrome --version # or inspect Chrome "About" settings page
```

SOLUTION ATTEMPTED:

Removed manually managed driver executables from the directory structure and system `PATH`. Updated Selenium to ≥ 4.6 and instantiated the driver without passing static `executable_path` arguments:

```
from selenium import webdriver
driver = webdriver.Chrome() # Automatically managed via Selenium Manager
```

RESULT AFTER THE FIX:

Selenium Manager queries and caches the exact matching driver binary for Chrome v126; the test browser session initializes cleanly.

SCREENSHOT OR LOG EVIDENCE:

```
DEBUG:selenium.webdriver.common.selenium_manager:Selenium Manager binary found at: ...
DEBUG:selenium.webdriver.common.selenium_manager:Driver path: ~/.cache/selenium/chromedriver/mac-
arm64/126.0.6478.127/chromedriver
```

One Preventive Recommendation: Eliminate static driver binaries in version control; rely on built-in Selenium Manager to maintain automated browser-driver parity.

C4 Browser binary not found

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: unknown error: cannot find Chrome binary
NoSuchDriverException: Unable to obtain driver for chrome
```

LIKELY ROOT CAUSE:

The browser executable is installed in a non-standard location (e.g., custom enterprise software drive or portable standalone folder) that is outside Selenium's default discovery search paths.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# Windows PowerShell check:
Test-Path "C:\Program Files\Google\Chrome\Application\chrome.exe"
# Linux / macOS check:
which google-chrome || which chromium-browser
```

SOLUTION ATTEMPTED:

Specified the explicit binary path using browser Options:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.binary_location = "D:/CustomApps/Google/Chrome/Application/chrome.exe"
driver = webdriver.Chrome(options=options)
```

RESULT AFTER THE FIX:

The driver successfully launches the designated browser instance from the specified non-standard file path.

SCREENSHOT OR LOG EVIDENCE:

```
INFO:selenium.webdriver.common.service:Starting ChromeDriver from ... with binary at D:/CustomApps/
Google/Chrome/Application/chrome.exe
Session ID: a82df910ce4 active.
```

One Preventive Recommendation: Define custom browser binary locations via environment variables (e.g., `CHROME_BINARY_PATH`) rather than hardcoded absolute file system paths.

C5 Permission or antivirus blocking

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: 'chromedriver' executable may have wrong permissions. (macOS/Linux)
PermissionError: [WinError 5] Access is denied / Windows Defender notification "Threat blocked"
```

LIKELY ROOT CAUSE:

The driver binary lacks POSIX executable bit permissions (`+x`), macOS Gatekeeper flagged the downloaded file with quarantine metadata, or corporate Endpoint Detection and Response (EDR) software quarantined the unsigned binary.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
ls -la ~/.cache/selenium/chromedriver/*/chromedriver
xattr -l /path/to/chromedriver # macOS Gatekeeper inspection
# Inspected Windows Defender / EDR quarantined items log
```

SOLUTION ATTEMPTED:

Applied proper execution permissions and cleared quarantine attributes on Unix-like systems:

```
chmod +x /path/to/chromedriver
xattr -d com.apple.quarantine /path/to/chromedriver
```

On enterprise Windows endpoints, requested an IT policy directory exclusion for `%LOCALAPPDATA%\Selenium`.

RESULT AFTER THE FIX:

Driver execution permissions verified; operating system permits the driver binary to spawn and listen on its assigned local port.

SCREENSHOT OR LOG EVIDENCE:

```
-rwxr-xr-x 1 user staff 15243120 Aug 28 10:00 /Users/user/.cache/selenium/chromedriver/.../
chromedriver
Status: Executable allowed by OS Gatekeeper.
```

One Preventive Recommendation: Incorporate automated permission checks (`chmod +x`) in CI/CD pipeline provisioning and coordinate standard directory exceptions with IT Security.

C6 Proxy or firewall failure

OBSERVED ERROR MESSAGE:

```
WebDriverException: Message: Unsuccessful request to https://googlechromelabs.github.io/chrome-for-testing/...: 407 Proxy Authentication Required  
ConnectionRefusedError: [Errno 111] Connection refused
```

LIKELY ROOT CAUSE:

Corporate or campus network proxies intercept and drop unauthenticated external HTTPS traffic initiated by Selenium Manager when fetching driver binaries.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
curl -I https://googlechromelabs.github.io/chrome-for-testing/  
echo $HTTP_PROXY $HTTPS_PROXY
```

SOLUTION ATTEMPTED:

Configured proxy environment variables for the shell and runtime:

```
export HTTP_PROXY="http://user:pass@proxy.corp.internal:8080"  
export HTTPS_PROXY="http://user:pass@proxy.corp.internal:8080"
```

Offline fallback: Pre-downloaded approved drivers into an offline storage cache and pointed driver service to the local binary via `Service(executable_path="/path/to/driver")`.

RESULT AFTER THE FIX:

Outbound driver discovery requests successfully authenticate through the network proxy; Selenium Manager downloads metadata cleanly.

SCREENSHOT OR LOG EVIDENCE:

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Connection: keep-alive  
DEBUG:selenium_manager: Driver downloaded successfully via corporate proxy.
```

One Preventive Recommendation: Establish standardized environment variables (`HTTP_PROXY` , `HTTPS_PROXY` , `NO_PROXY`) or maintain an internal binary mirror (Artifactory/Nexus).

C7 SSL certificate error during pip install

OBSERVED ERROR MESSAGE:

```
pip._vendor.urllib3.exceptions.SSLError: [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed:  
self signed certificate in certificate chain (_ssl.c:1006)
```

LIKELY ROOT CAUSE:

Corporate TLS deep packet inspection proxies (e.g., Zscaler) re-sign traffic using an enterprise Root CA certificate that is not bundled inside Python's default `certifi` certificate store.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
python -c "import certifi; print(certifi.where())"  
# Inspected system root certificate trust store for corporate root CA
```

SOLUTION ATTEMPTED:

Configured pip to trust the organization's official Root CA certificate bundle:

```
pip config set global.cert /etc/ssl/certs/corp-root-ca.crt  
# Or via environment variables:  
export REQUESTS_CA_BUNDLE=/etc/ssl/certs/corp-root-ca.crt  
export SSL_CERT_FILE=/etc/ssl/certs/corp-root-ca.crt
```

RESULT AFTER THE FIX:

`pip install` performs full TLS verification against PyPI via the corporate certificate chain and installs packages securely.

SCREENSHOT OR LOG EVIDENCE:

```
Collecting selenium
  Downloading selenium-4.22.0-py3-none-any.whl (10.2 MB)
Installing collected packages: selenium
Successfully installed selenium-4.22.0
```

One Preventive Recommendation: Configure the approved Root CA path in global `pip.ini` / `pip.conf` instead of bypassing security via `--trusted-host` or insecure flags.

C8 Browser opens and immediately closes

OBSERVED ERROR MESSAGE:

```
Process finished with exit code 0 (Browser window appears and terminates in < 500ms before assertions or inspection can occur)
```

LIKELY ROOT CAUSE:

The WebDriver instance was declared within a local function scope and immediately garbage-collected upon function return, or the standalone Python script reached EOF without structured test runner lifecycle management.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# Code inspection identified local scope de-allocation:
def initialize():
    driver = webdriver.Chrome() # Object destroyed upon return!
initialize()
```

SOLUTION ATTEMPTED:

Employed structured test runner fixtures (`pytest` with `yield` teardown) or enabled the experimental detach option during manual debugging:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_experimental_option("detach", True)
driver = webdriver.Chrome(options=options)
driver.get("https://example.com")
```

RESULT AFTER THE FIX:

The browser session remains active throughout test execution and only shuts down when explicit teardown logic is invoked.

SCREENSHOT OR LOG EVIDENCE:

```
2026-08-28 10:00:00 [INFO] Navigation complete. Session ID: 9f8a3 active.
Process finished with exit code 0 (Browser session preserved for inspection).
```

One Preventive Recommendation: Use standard test lifecycle frameworks (e.g., Pytest fixtures with `try...finally` / `yield driver` teardowns) instead of procedural ad-hoc scripts.

C9 Element not found

OBSERVED ERROR MESSAGE:

```
selenium.common.exceptions.NoSuchElementException: Message: no such element: Unable to locate element:
{"method":"css_selector","selector":"#submit-btn"}
```

LIKELY ROOT CAUSE:

Single Page Application (SPA) asynchronous rendering delays, the element is nested within an `<iframe>` / Shadow DOM tree, or the locator relied on an unstable auto-generated dynamic ID.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
# DOM Inspection checks:
# 1. Verified whether element resides inside an <iframe> or #shadow-root
# 2. Checked network tab for pending AJAX / Fetch requests
```

SOLUTION ATTEMPTED:

Replaced raw locator calls with `WebDriverWait` explicit waits and resilient semantic locators:

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Frame switch if applicable:
# WebDriverWait(driver, 10).until(EC.frame_to_be_available_and_switch_to_it((By.ID, "frame_id")))

wait = WebDriverWait(driver, 10)
btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "button[data-testid='submit-btn']")))
btn.click()
```

RESULT AFTER THE FIX:

The driver dynamically polls the DOM until conditions match; element is located and clicked without timing exceptions.

SCREENSHOT OR LOG EVIDENCE:

```
DEBUG:selenium.webdriver.remote.remote_connection:POST /session/.../element {"using": "css_selector",
"value": "button[data-testid='submit-btn']"}
DEBUG:selenium.webdriver.remote.remote_connection:Remote response: status=200, value={'ELEMENT': 'f.
84B6...'}
```

One Preventive Recommendation: Adopt explicit `WebDriverWait` with `expected_conditions` and mandate dedicated test automation attributes (e.g., `data-testid`) across front-end components.

C10 Headless works differently

OBSERVED ERROR MESSAGE:

```
ElementClickInterceptedException: Message: element click intercepted: Element <button>...</button> is
not clickable at point (120, 45). Other element would receive the click.
```

LIKELY ROOT CAUSE:

Default headless mode initializes at a smaller viewport (e.g., 800x600), triggering responsive mobile stylesheets that collapse navigation menus into hamburgers or overlap click targets.

DIAGNOSTIC COMMANDS OR CHECKS PERFORMED:

```
print(driver.get_window_size())
driver.save_screenshot("headless_failure.png")
with open("headless_source.html", "w") as f:
    f.write(driver.page_source)
```

SOLUTION ATTEMPTED:

Configured Chrome's modern headless engine (`--headless=new`), set standard 1080p viewport dimensions, and supplied a standard desktop User-Agent string:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless=new")
options.add_argument("--window-size=1920,1080")
options.add_argument("--user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/126.0.0.0 Safari/537.36")

driver = webdriver.Chrome(options=options)
```

RESULT AFTER THE FIX:

The DOM renders the exact desktop viewport layout, resolving intercepted clicks and matching headed execution results.

SCREENSHOT OR LOG EVIDENCE:

INFO: Screen resolution logged at 1920x1080.
Headless diagnostic screenshot confirmed full desktop layout. All assertions PASSED.

One Preventive Recommendation: Always standardize browser resolution in base test configurations using explicit `--window-size=1920,1080` arguments across both headed and CI headless environments.

7. Part D – Mini Automation Task (15 marks)

Write a Selenium script that completes all of the following on a public practice site approved by your instructor:

- Open the site and verify the page title.
- Locate and interact with at least three different element types.
- Use at least two locator strategies.
- Use an explicit wait for a meaningful condition.
- Capture a screenshot after successful interaction.
- Handle one expected failure using exception handling and a useful error message.
- Close the session safely even if an error occurs.

Add comments explaining why each locator and wait strategy was selected. Do not automate login to a real personal account.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import NoSuchElementException,
TimeoutException
import time

driver = webdriver.Chrome()

# Explicit wait:
# We use WebDriverWait instead of time.sleep() because Selenium
# should continue as soon as the required condition is satisfied.
wait = WebDriverWait(driver, 10)

try:

    driver.get("https://the-internet.herokuapp.com/add_remove_elements/")

    # Verify the page title
    expected_title = "The Internet"

    if driver.title == expected_title:
        print("PASS: Page title is correct.")
    else:
        print(f"FAIL: Expected '{expected_title}', got '{driver.title}'")

    # CSS Selector is used here because the Add Element button
    # has a distinctive class and CSS provides a concise locator.
    add_button = wait.until(
        EC.element_to_be_clickable(
            (By.CSS_SELECTOR, "button[onclick='addElement()']")
        )
    )

    add_button.click()

    print("PASS: Add Element button clicked.")
```

```

# The Delete button appears only after clicking Add Element.
# We explicitly wait for it to become visible instead of
# assuming it is immediately available.
delete_button = wait.until(
    EC.visibility_of_element_located(
        (By.CLASS_NAME, "added-manually")
    )
)

print("PASS: Delete button appeared.")

delete_button.click()

print("PASS: Delete button clicked.")


driver.get("https://the-internet.herokuapp.com/dynamic_controls")


# ID is used because "checkbox" is a unique and stable
# identifier on this practice page.
checkbox = wait.until(
    EC.presence_of_element_located(
        (By.ID, "checkbox")
    )
)

print("PASS: Checkbox located.")

checkbox.click()

print("PASS: Checkbox clicked.")


# XPath is used here to demonstrate a second locator strategy.
# We locate the text input based on its type attribute.
text_box = wait.until(
    EC.visibility_of_element_located(
        (By.XPATH, "//input[@type='text']")
    )
)

text_box.clear()
text_box.send_keys("Selenium with Python")

print("PASS: Text entered into input field.")


screenshot_path = "selenium_success.png"

driver.save_screenshot(screenshot_path)

print(f"PASS: Screenshot saved as {screenshot_path}")

```

```

try:

    # This element intentionally does NOT exist.
    # We catch the expected failure rather than allowing
    # the entire test to crash.
    missing_element = driver.find_element(
        By.ID,
        "element-that-does-not-exist"
    )

except NoSuchElementException:
    print(
        "EXPECTED FAILURE: Could not find element with ID "
        "'element-that-does-not-exist'."
    )

except TimeoutException as e:

    print(
        f"ERROR: Selenium waited 10 seconds but the expected "
        f"condition was not satisfied.\nDetails: {e}"
    )

except Exception as e:

    print(f"UNEXPECTED ERROR: {e}")

finally:

    # finally guarantees that the browser is closed even if
    # an error occurs somewhere inside the try block.
    driver.quit()

    print("Browser session closed safely.")

```

R1. Which installation or execution issue required the most reasoning, and how did you isolate its cause?

The issue that required the most reasoning was the `ModuleNotFoundError: No module named 'selenium'` error. I first checked whether the virtual environment was active, which was indicated by `(venv)` in the terminal. I then realized that Selenium had not been installed inside that specific virtual environment. I isolated the cause by running `python -m pip install selenium` and verifying the installation with `python -m pip show selenium`. After that, the Python script was able to import Selenium successfully.

I also encountered a syntax error while trying to locate an Amazon element. I had copied the entire HTML element into `find_element()` and tried to use multiple class names with `By.CLASS_NAME`. I identified that `By.CLASS_NAME` accepts only one class name and changed the locator to a CSS selector, which supports multiple classes.

R2. What would you change in your setup to make future Selenium projects easier to reproduce?

For future Selenium projects, I would create a separate Python virtual environment for each project and maintain a `requirements.txt` file containing the required packages and their versions. I would also keep the project structure organized, for example with separate folders for tests, page objects, utilities, and configuration files.

I would use commands such as `python -m pip install -r requirements.txt` to recreate the environment on another computer. I would also keep the Python and Selenium versions documented so that other users can reproduce the same setup more easily.

R3. Which source was most useful, and how did you verify that its advice applied to your Selenium version?

The official Selenium documentation was the most useful source because it provides information about WebDriver, locators, waits, and browser automation directly from the Selenium project. I verified that the examples applied to my setup by checking the installed Selenium version using `python -m pip show selenium` and then testing the relevant commands in my Python virtual environment.

I also verified the advice practically by running a simple Selenium script that opened Chrome, navigated to a website, located elements, and interacted with them. This confirmed that the documented Selenium APIs worked correctly in my installed version.